

1

代码风格方面：

简化了数据处理部分的冗长代码，进行了系统编号，使项目结构更加清晰。
右图为处理电梯变量的期中代码与期末代码对比。

2

变量处理方面：

- (1) 将装修变量、建筑结构变量由哑变量修改为离散数值变量
- (2) 将小区规模变量与楼栋总数变量修改为分箱哑变量
- (3) 将交易时间变量修改为只提取年、月，并引入距上次交易时间的新变量

3

特征工程方面：

系统分为三类变量：城市哑变量、平方项、城市交互项
既有效避免了过拟合（无三次项、很少有除城市外的哑变量），又充分捕捉了关键的非线性效应与群体异质性，确保了模型的表达能力和预测精度。

4

文本数据处理方面：

利用经纬度计算距市中心的距离，作为预测房价的一个有效因子。

```
# 1.2.10 电梯变量处理（购房训练集和测试集）

def process_elevator_variables(df, dataset_name=""):
    """处理电梯变量，创建哑变量并确保数据类型正确"""
    if dataset_name:
        print(f"{dataset_name}电梯分布:")
    else:
        print("电梯分布:")

    print(df['配备电梯'].value_counts())
    print(f"缺失值: {df['配备电梯'].isna().sum()}")

    # 删除已存在的电梯哑变量列
    elevator_columns_to_drop = [col for col in df.columns if col.startswith('elevator_')]
    df = df.drop(columns=elevator_columns_to_drop)

    # 创建电梯哑变量，直接创建为浮点数
    df['elevator_yes'] = df['配备电梯'].map({'无': 0.0, '有': 1.0})

    # 处理缺失值 - 将原始数据中的缺失值反映到哑变量中
    if df['配备电梯'].isna().any():
        na_mask = df['配备电梯'].isna()
        df.loc[na_mask, 'elevator_yes'] = np.nan

    # 打印结果
    print(f"\n{dataset_name}电梯哑变量分布:")
    valid_count = df['elevator_yes'].notna().sum()
    true_count = (df['elevator_yes'] == 1.0).sum()
    false_count = (df['elevator_yes'] == 0.0).sum()
    na_count = df['elevator_yes'].isna().sum()
    print(f"有电梯: {true_count}个")
    print(f"无电梯: {false_count}个")
    print(f"缺失值: {na_count}个")

    return df

# 处理购房训练集
df_price_train = process_elevator_variables(df_price_train, "购房训练集")

# 处理购房测试集
df_price_test = process_elevator_variables(df_price_test, "购房测试集")
```



```
# 1.2.11 elevator variable processing

def process_elevator_variables(df):
    df['elevator_yes'] = df['配备电梯'].map({'无': 0.0, '有': 1.0}).fillna(0.0)
    return df

df_price_train = process_elevator_variables(df_price_train)
df_price_test = process_elevator_variables(df_price_test)
```

1

集成学习算法方面

尝试了以下三种算法
Random Forest (Best)
Xgboost
LightGBM

House Price Predicting Results (lnPrice)				
Metrics	In sample rmse	Out of sample rmse	Cross-validation rmse	Kaggle Score
ridge	0.3494	0.3520	0.3503	59.1
Random Forest	0.1085	0.1756	0.1829	72.6
Xgboost	0.1458	0.1718	0.1738	72.1
LightGBM	0.1476	0.1675	0.1712	71.8
Keras ANN	0.2027	0.2240	0.2386	68.6
Pytorch ANN	0.1999	0.2165	0.2185	69.7

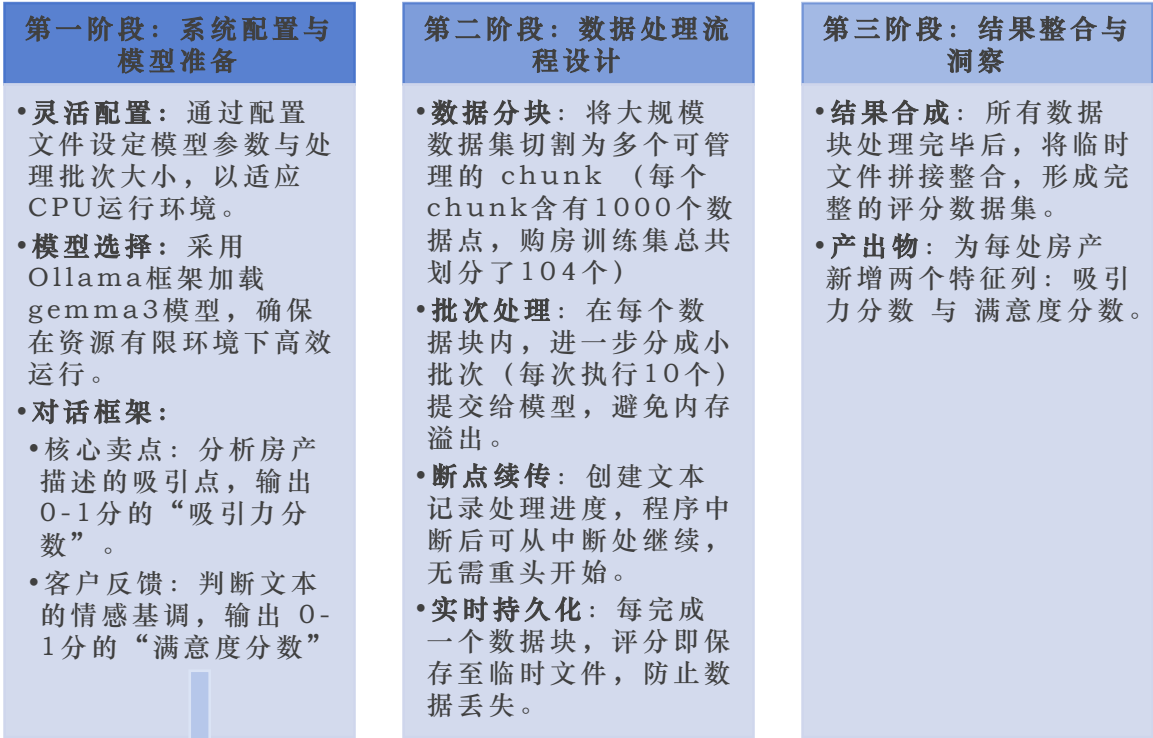
2

深度学习算法方面

尝试了以下两种种算法
Keras ANN
Pytorch ANN

Rent Price Predicting Results (lnPrice)				
Metrics	In sample rmse	Out of sample rmse	Cross-validation rmse	Kaggle Score
ridge	0.3024	0.3043	0.3031	59.1
Random Forest	0.3051	0.3071	0.3058	72.6
Xgboost	0.1240	0.1909	0.1961	72.1
LightGBM	0.1595	0.1904	0.1914	71.8
Keras ANN	0.2041	0.2297	0.2422	68.6
Pytorch ANN	0.2058	0.2220	0.2266	69.7

(a) 利用本地大语言模型（LLM），将户型优势与客户反馈等非结构化文本，自动转化为可量化的“吸引力”与“满意度”评分。



Prompt:
请根据以下客户反馈，评估其整体满意度和情感倾向，并为其打分。
注意：反馈中可能包含口语化、转折或矛盾表述，请综合判断整体基调。
返回一个有效的JSON对象，必须包含以下键值：
{
 "score": 0.0到1.0之间的数字，其中：
 0.0-0.3代表"负面为主"（如抱怨设施、物业、环境等），
 0.3-0.7代表"中性或混合"（有褒有贬，或描述平淡），
 0.7-1.0代表"正面为主"（如称赞体验、设施、环境等）
}
反馈: "{text}"

(b) Optuna智能优化随机森林模型

- 智能参数探索：自动搜索最优参数组合
- 贝叶斯优化：基于历史试验结果智能采样
- 50轮迭代：全面覆盖参数空间

Metrics\Models	Random Forest	Optuna优化	进步率
In sample rmse	0.1085	0.0603	44.39%
Out of sample rmse	0.1756	0.1578	10.14%
Cross-validation rmse	0.1829	0.1643	10.17%
Kaggle Score	72.6	74.5	2.62%

Metrics\Models	Random Forest	Optuna优化	进步率
In sample rmse	0.3051	0.1013	66.80%
Ot of sample rmse	0.3071	0.1892	38.39%
Cross-validation rmse	0.3058	0.1938	36.63%
Kaggle Score	72.6	74.5	2.62%